

React

The library for web and native user interfaces

Contenido

INTRODUCCION 2
DIFERENCIAS ENTRE S.P.A Y M.P.A 2
DEFINICIÓN DE REACT 3
RUTA DE APRENDIZAJE 4
LOS HOOK EN REACT 4
CICLO DE VIDA DE UN HOOK EN REACT 5
HOOKS IMPRESCINDIBLES 6
OTROS HOOKS 8
PROYECTO DE EJEMPLO 001: CONSUMIR API RICK Y MORTY 8
INSTALACIÓN Y CREACIÓN DE PROYECTO USANDO WEBPACK 8
INSTALACIÓN REACT USANDO EL BUNDLER VITE 9
APROPIACION DE CONOCIMIENTO 15

INTRODUCCION

En el mundo actual, donde la experiencia del usuario es fundamental, **las Single Page Applications (SPA)** se han convertido en una opción cada vez más popular para el desarrollo de aplicaciones web, ya que les da un aspecto interactivo y fluido, con características similares a las de las apps nativas.

"Una single page application (SPA) o aplicación de una sola página, es un tipo de aplicación web que carga una única página inicial desde el servidor y luego actualiza dinámicamente su contenido, mediante las interacciones del usuario, sin necesidad de cargar páginas adicionales del servidor."

"Las SPA, en lugar de realizar solicitudes tradicionales al servidor para obtener nuevas páginas HTML, utiliza tecnologías como JavaScript, Ajax y frameworks de frontend como Angular, React o Vue.js para mostrar datos sin necesidad de recargar toda la página.

Su característica principal es que proporciona una experiencia de usuario fluida y rápida, similar a una aplicación de escritorio, pues los cambios de contenido ocurren de forma instantánea y sin interrupciones. **Esto se logra mediante la manipulación del DOM (Modelo de Objetos del Documento) y el enrutamiento interno en la app**. Al cargar la página inicial, se descargan los recursos necesarios, como archivos HTML, CSS y JavaScript, con los que se construye y actualiza la interfaz de usuario.

Las interacciones de los consumidores, como hacer clic en un enlace o enviar un formulario, activan acciones en JavaScript que **actualizan solo las partes relevantes de la página**, sin tener que recargar todo el sitio nuevamente. También, las SPA se emplean en el desarrollo web moderno, debido a su capacidad para crear experiencias de usuario ricas y altamente interactivas y aplicaciones más rápidas y eficientes al reducir la carga en el servidor.

DIFERENCIAS ENTRE S.P.A Y M.P.A

SINGLE PAGE APPLICATION	MULTI PAGE APPLICATION
Todo el código de la aplicación se carga inicialmente en el navegador y las interacciones posteriores se realizan mediante la manipulación del DOM y actualizaciones dinámicas del contenido.	una MPA carga páginas completas adicionales desde el servidor en respuesta a las acciones del usuario.
La navegación dentro de la aplicación se realiza a través de cambios de estado y actualizaciones	La navegación implica cargar páginas completamente nuevas desde el servidor, lo que

dinámicas en la misma página, por lo general, con enrutamiento interno.	puede resultar en una experiencia de usuario más lenta debido al tiempo de carga adicional.
La experiencia de usuario es más fluida y rápida, ya que las actualizaciones de contenido ocurren de manera instantánea sin necesidad de recargar toda la página.	La experiencia puede sentirse más fragmentada, pues se produce un parpadeo entre las transiciones de las páginas adicionales.
Son más eficientes en términos de uso de ancho de banda y recursos del servidor, solo se cargan los datos necesarios y no se transmiten páginas HTML completas con cada solicitud.	Requieren de una comunicación constante con el servidor para obtener nuevas páginas HTML y actualizar el contenido completo de la página.
El desarrollo de una Single Page Application puede ser más complejo debido a que utiliza frameworks y bibliotecas frontend, así como manejar la lógica de enrutamiento y la gestión del estado por el lado del cliente. Además, las SPA suelen tener una estructura modular, lo que puede facilitar el mantenimiento y la escalabilidad.	El desarrollo de las MPA es más tradicional y basado en el servidor.

DEFINICIÓN DE REACT

React es una librería para crear interfaces de usuarios con código abierto, es decir, que está a disposición de cualquier programador para que use sus recursos e incluso haga colaboraciones. Esta biblioteca fue lanzada en el año 2013, por los desarrolladores de Facebook (Meta), con base en el lenguaje JavaScript.

Actualmente es una de las herramientas más utilizadas para crear páginas web (SPA) porque su uso es bastante amigable y fácil de aprender. Sobre todo, si ya se tienen conocimientos del lenguaje JavaScript, ya que la sintaxis que usa es muy similar. Cabe mencionar que necesita otro software para complementar las funciones de un sitio web, **porque no es esencialmente un framework** (tal como angular).

Por supuesto, es la base de la programación de **Facebook, Instagram y WhatsApp**; y varias compañías internacionales como **Tesla, Netflix, Twitter, Uber, Walmart, Shopify, AirBnb**, entre otras. Asimismo, React cuenta con su repositorio en la plataforma de GitHub, donde tanto desarrolladores internos y externos comparten sus modificaciones sugeridas al código.

Además de la configuración de nuevas páginas web, esta librería de JavaScript sirve para crear aplicaciones móviles con la complementación del **framework React Native.** Y, de igual modo, para crear aplicaciones de realidad virtual con la ayuda del **framework React 360**.

RUTA DE APRENDIZAJE

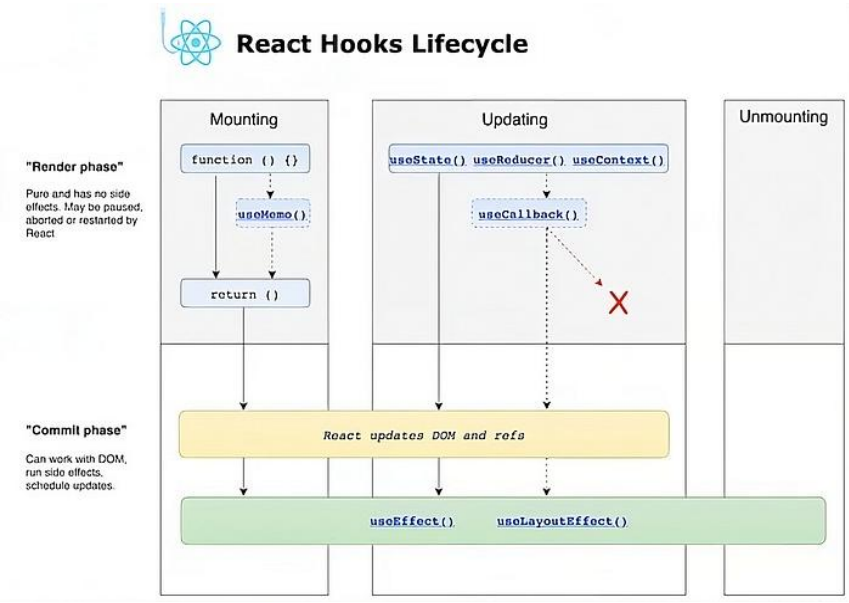
INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO	
COMPONENTES	FUNCIONAMIENTO DEL REACT INSTALACIÓN Y PRIMEROS PASOS CON VITE HOLA MUNDO PRIMEROS COMPONENTES ESTRUCTURAS DE DATOS EN REACT PASO DE VALORES ENTRE COMPONENTES ACTUALIZACIÓN DE ESTADO COMPONENTES(USESTATE) RENDERIZADO CLASES DINÁMICAS RENDER LISTAS RENDER CONDICIONADO HOOKS: USEEFFECT, USESTATE, USEMEMO,
ROUTING O ENRUTAMIENTO	
PAGINACION, FILTRADO Y ORDENAMIENTO	
FORMULARIOS	
LLAMANDO SERVICIOS DE BACKEND	
DESPLIEGUE	

LOS HOOK EN REACT

Los hooks de React son funciones que nos permiten acceder al estado y a las funcionalidades del ciclo de vida de los componentes funcionales. Con los hooks, podemos extraer la lógica con estado de los componentes para probarla de forma independiente y reutilizarla. Los hooks nos permiten reutilizar la lógica con estado sin modificar la jerarquía de componentes, lo que facilita compartir hooks entre varios componentes o con la comunidad. También existen otras reglas que se deben seguir al usar hooks.

- Los hooks solo se ejecutan en el ámbito de los componentes y siempre se codifican en el nivel superior de la función REACT. Todos los hooks comienzan con la palabra "use".

CICLO DE VIDA DE UN HOOK EN REACT



En React, el término “ciclo de vida” o lifecycle, se refiere a las diferentes fases por los que pasa un componente desde su creación hasta su destrucción. Estas fases son representadas por una serie de métodos que se ejecutan en momentos específicos durante la vida útil del componente.

El ciclo de vida de un componente nos ayuda a tomar decisiones lógicas dependiendo de su fase, como por ejemplo nos ayuda a saber cuándo realizar una petición HTTP o cuando poder limpiar datos en la UI.

El ciclo de vida de un componente se divide principalmente en 3 fases.

1. Mounting (Montado): Fase que se gatilla cuando el componente es montado en la UI.
2. Updating (Actualizado): Fase que gatilla cuando el componente sufre un cambio en su estado o props.
3. Unmounting (Desmontado): Fase que se gatilla cuando el componente es desmontado de la UI.

HOOKS IMPRESCINDIBLES

De estado: El Estado permite a un componente "recordar" información como el input del usuario. Por ejemplo, un componente de formulario puede usar el estado para almacenar el valor de entrada, mientras que un componente de galería de imágenes puede usar el estado para almacenar el indicé de la imagen seleccionada.	**Usestate()**	declara un estado variable que puedes actualizar directamente.
	Usereduce()	declara un estado variable con la lógica de actualización dentro de una función reductora.
De contexto: El Contexto permite a un componente recibir información desde un componente padre distante sin pasarsela como props. Por ejemplo, tu componente en el nivel más alto de tu aplicación puede pasar el tema de interfaz de usuario actual a todos los componentes debajo, no importa que tan profundo estén	Usecontext()	lee y se suscribe a un Contexto.
De Refs: Los Refs permiten a un componente conserva algo de información que no es usada para el renderizado, como un nodo del DOM or el ID de un timeout. A diferencia del estado, actualizar un ref no no vuelve a renderizar tu componente. Los refs son una "puerta de escape" del paradigma de React. Son útiles cuando	Useref()	declara un ref. Puedes conservar cualquier valor en él, pero más frecuentemente se utiliza para conservar un node del DOM.
	Useimperativehandle()	ye permite personalizar el ref expuesto por tu componente. Es raramente utilizado.

necesitas trabajar con sistemas que no son de React, como son las APIs integradas del navegador		

De Efecto: Los Efectos permiten a un componente conectarse y sincronizarse con sistemas externos. Esto incluye lidiar con la red, el DOM del navegador, animaciones, widgets escritos usando una biblioteca UI distinta, y otro código que no sea de React..	useEffect conecta un componente a un sistema externo.
	useLayoutEffect se activa antes de que el navegador vuelve a pintar la pantalla. Puedes medir la maquetación aquí.
	useInsertionEffect se activa antes de que React realice cambios al DOM. Las bibliotecas pueden insertar CSS dinámico aquí.
De rendimiento: Una forma común de optimizar el rendimiento del re-renderizado es saltarse el trabajo innecesario. Por ejemplo, puedes decirle a React que reutilice cálculos que están en la caché o que se salte un re-renderizado si los datos no han cambiado desde el renderizado anterior.	useMemo te permite almacenar en caché el resultado de un cálculo costoso.
	useCallback te permite almacenar en caché la definición de una función antes de pasarla a un componente optimizado.
	useTransition te permite marcar un estado de transición como no bloqueante y permite a otras actualizaciones interrumpirlo.
	useDeferredValue te permite aplazar la actualización de una parte no critica de la UI y permite a las otras partes actualizarse primero.

OTROS HOOKS

- Estos Hooks son mayormente útiles para los autores de bibliotecas y no son comúnmente utilizados para el código de una aplicación.
    - useDebugValue te permite personalizar la etiqueta que las herramientas de desarrollo de muestran para tu Hook personalizado.
    - useId permite a un componente se asocie a sí mismo un identificador único. Típicamente es usado con APIs de accesibilidad.
    - useSyncExternalStore permite a un componente suscribirse a una store externo.
    - useActionState allows you to manage state of actions.

PROYECTO DE EJEMPLO 001: CONSUMIR API RICK Y MORTY

Es una aplicación que permite la gestión del api: https://rickandmortyapi.com/

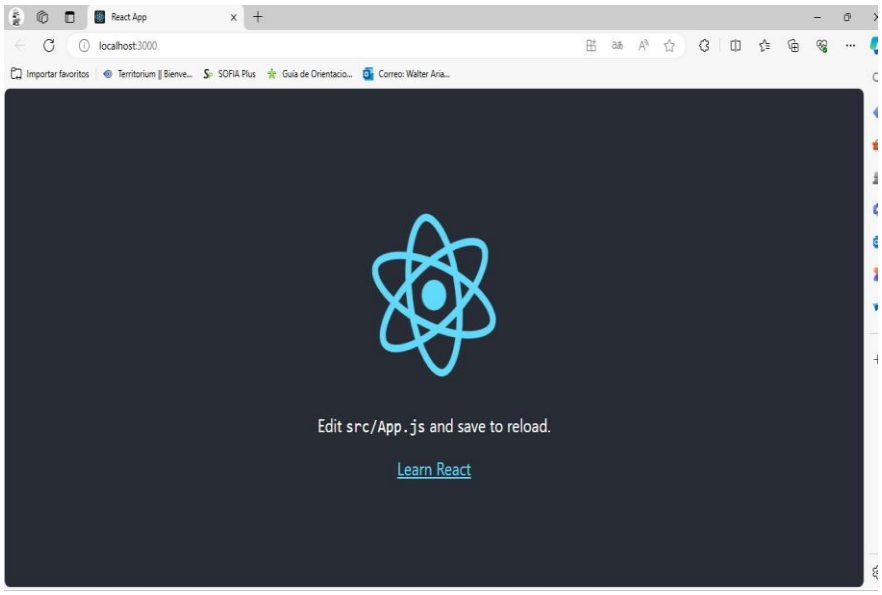
Rick y morty es una serie de dibujos animados, la api contiene información de los personajes y los episodios.

usaremos y BOOTSTRAP como framework CSS (o Tailwind) además de un bundler (empaquetador) en este caso será vite

INSTALACIÓN Y CREACIÓN DE PROYECTO USANDO WEBPACK

1. Crear un subdirectorio denominado rickmortyweb
2. Abrimos el subdirectorio con la consola (powershell, cmd, o git bash)
3. Ejecutamos el comando en consola “npm create vite@latest” el cual Crea un proyecto react con la librería npm
4. Si sale error se tiene que actualizar el npm con estos comandos:
    - npm uninstall -g create-react-app
    - npm install -g create-react-app
    - npx create-react-app <*nombreproyecto*>
5. Ingresamos a la carpeta creada con el comando cd rickmortyweb
6. abrimos el editor vs code con el comando “code . ”

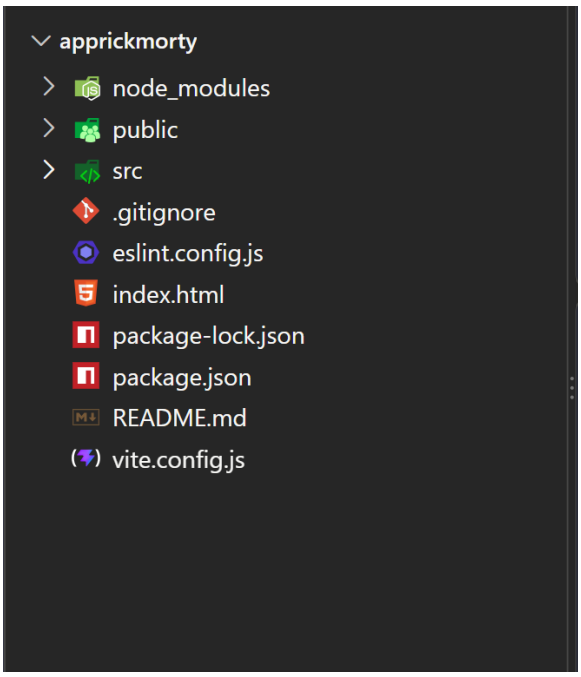
7. iniciamos el proyecto con el comando " npm start " y nos enviara a una página local en el navegador
8. (La carpeta a mostrar al cliente es la carpeta llamada public)
9. (La puerta de entrada son los archivos app.js y app.css se encuentran en la carpeta src)



¡Instalación de react exitosa!

INSTALACIÓN REACT USANDO EL BUNDLER VITE

1. Si vamos a usar vite como bundler o empaquetador el comando es:
   npm create vite@latest **<nombre-de-mi-proyecto>***rickmortyapp*
2. Seleccionamos react, javascript y al finalizar cuando cargue el proyecto cancelamos con la combinación CTRL+C
3. Nos ubicamos en el subdirectorio *rickmortyapp y con el comando code . abrimos el editor vscode*
4. Al ingresar observamos esta organización de subdirectorios



Instalaremos bootstrap como framework css dentro de nuestra app, para ello abrimos la consola integrada de vscode y ejecutamos el comando : npm i bootstrap@5.3.8

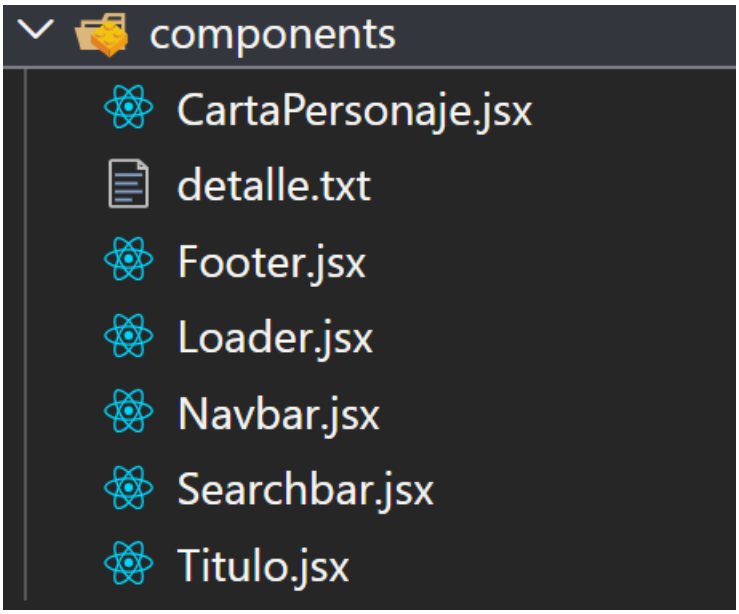
Posteriormente editamos el archivo main.jsx agregando estas líneas en la cabecera

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "bootstrap/dist/js/bootstrap.bundle.js"
import App from "./App.jsx";
```

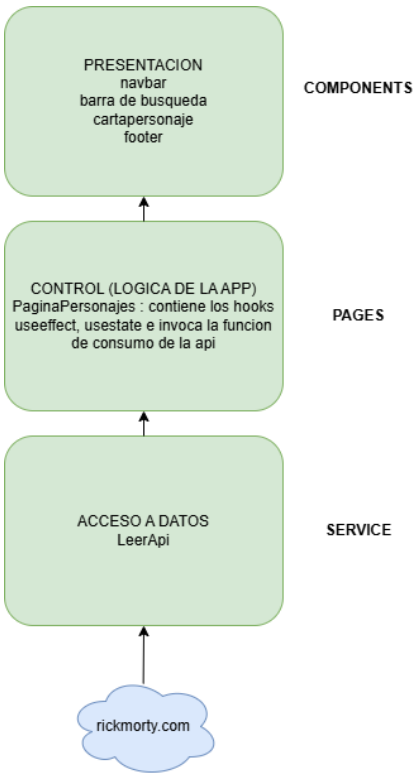
Dentro del subdirectorio SRC (en donde se construirá el código de la app) creamos la siguiente estructura de directorios o scaffolding:



La carpeta componentes contendrá todos los elementos reusables de la UI de nuestra app con la guía de su instructor construya todos estos componentes:



El esquema de carga de la app será el siguiente:



Cree el componente CartaPersonaje.jsx y escriba este código:

```
export default function CartaPersonaje({ character }) {
  return (
    <div className="card h-100 shadow-sm">
      <img src={character.image} className="card-img-top" alt="..."></img>

      <div className="card-body">
        <h5>{character.name}</h5>
        <p>{character.species}</p>
        <span className="badge bg-success">{character.status}</span>

        <button className="btn btn-primary w-100 mt-2">Ver detalle</button>
      </div>
    </div>
  );
}
```

Dentro de la carpeta services cree el componente LeerApi.jsx y escriba este código:

const API_URL = "https://rickandmortyapi.com/api/character";

```
export const obtienePersonajes = async (name = "") => {
  let url = API_URL;

  if (name) {
    url += `/?name=${name}`;
  }

  const response = await fetch(url);

  if (!response.ok) {
    throw new Error("Error al consultar API");
  }

  return await response.json();
};
```

Finalmente, dentro del subdirectorio Pages cree el componente PaginaPersonajes.jsx y adicione este código

```
import { useEffect, useState } from "react";

import CartaPersonaje from "../components/CartaPersonaje";
import { obtienePersonajes } from "../services/LeerApi";

export default function PaginaPersonajes() {
  const [characters, setCharacters] = useState([]);

  useEffect(() => {
    loadCharacters();
  }, []);

  const loadCharacters = async () => {
    try {
      const data = await obtienePersonajes();

      setCharacters(data.results);
    } catch (error) {
      console.error(error);
    }
  };

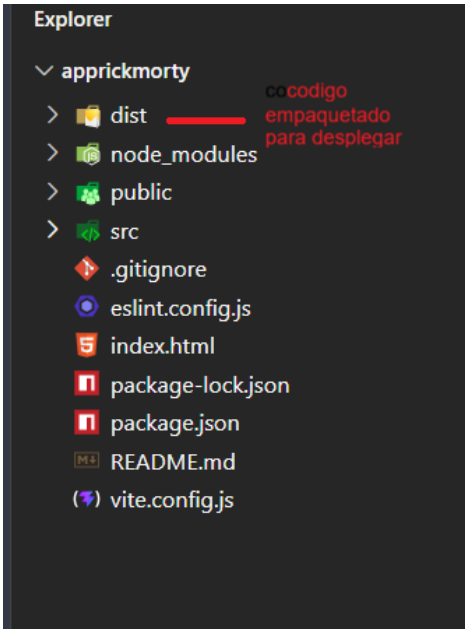
  return (
    <div className="row">
```

```
      {characters.map((character) => (
        <div key={character.id} className="col-md-3 mb-4">
          <CartaPersonaje character={character} />
        </div>
      ))}
    </div>
  );
}
```

Abra la consola integrada de vscode e inicie el servicio con el comando **npm run dev,** verifique en la web la carga del sitio de forma correcta, abra la consola y estudie los componentes que constituyen la aplicación.

FASE DE PRODUCCION

Para desplegar el proyecto en producción (para consumo de los usuarios finales del sistema ejecutamos en consola: **npm run build**



El subdirectorio dist lo desplegamos a producción en nuestro hosting de preferencia (netlify)

APROPIACION DE CONOCIMIENTO

Usando el mismo estilo de trabajo aplicado anteriormente para la api: https://jsonplaceholder.typicode.com/ cree una app que consuma y renderice el recurso o endpoint **/users**

Reto adicional: intente inyectar un framework diferente a boostrap en su proyecto por ejemplo materialize, tailwind, etc.